MACHINE LEARNING

DECISION TREES

Mathematical Foundations, Algorithms and From-Scratch Python

Three algorithms, three distinct split philosophies

ID3 | C4.5 | CART

What this guide is designed to teach

Understand the mathematics behind a split; calculate a split by hand; construct a tree one node at a time; implement the algorithms using only core Python; and explain why ID3, C4.5 and CART are not interchangeable.

Self-study and examination-oriented notes

Includes solved numerical problems, pseudocode, implementation walkthroughs and a practice sequence.

Contents

1 How to use these notes **3**
1.1 The fixed format used for every solved problem . . . 3
1.2 What you should be able to do after studying . . . 3

2 Shared mathematical foundations **3**
2.1 Supervised learning setup . . . 3
2.2 What a decision tree learns . . . 4
2.3 Class proportions . . . 4
2.4 Entropy: uncertainty in a class distribution . . . 4
2.5 Weighted impurity after a split . . . 5
2.6 Information gain . . . 5
2.7 Split information and gain ratio . . . 5
2.8 Gini impurity . . . 6
2.9 Regression impurity: squared error and variance . . . 6
2.10 Continuous features and candidate thresholds . . . 7
2.11 Greedy recursive splitting . . . 7
2.12 Stopping, leaf predictions and overfitting . . . 7

3 ID3: Iterative Dichotomiser 3 **7**
3.1 Purpose and defining idea . . . 8
3.2 How ID3 handles an attribute . . . 8
3.3 ID3 algorithm, step by step . . . 8
3.4 Pseudocode . . . 8
3.5 Solved numerical problem 1: selecting the ID3 root . . . 10
3.6 Solved numerical problem 2: completing the ID3 tree . . . 11
3.7 What ID3 does well and where it is limited . . . 11
3.8 From-scratch ID3 implementation . . . 11
3.8.1 Small runnable example: Play Tennis . . . 13
3.8.2 How to read the ID3 code . . . 14

4 C4.5: the gain-ratio decision tree **14**
4.1 Purpose and relationship to ID3 . . . 14
4.2 Gain ratio: why normalize information gain? . . . 15
4.3 Categorical versus continuous attributes in C4.5 . . . 15
4.4 Missing values and pruning: conceptual treatment . . . 15
4.5 Solved numerical problem 3: calculating gain ratio on the Play Tennis data . . . 16
4.6 Solved numerical problem 4: choosing a numeric threshold in C4.5 . . . 17
4.7 From-scratch educational C4.5 implementation . . . 17
4.7.1 Small runnable example: a numeric feature . . . 20
4.7.2 How to trace the C4.5 code . . . 21

5 CART: Classification and Regression Trees **21**
5.1 Purpose and defining structure . . . 21
5.2 CART for classification . . . 21
5.3 How CART creates binary splits . . . 22
5.4 Solved numerical problem 5: selecting a categorical CART split using Gini . . . 23
5.5 Solved numerical problem 6: numeric threshold using CART Gini . . . 24
5.6 CART for regression . . . 24
5.7 Solved numerical problem 7: a CART regression split . . . 25
5.8 Cost-complexity pruning in CART . . . 25
5.9 From-scratch CART implementation: classification and regression . . . 26
5.9.1 Small runnable examples . . . 30

5.9.2 Implementation boundaries for this CART code 30

6 A direct comparison: do not mix the algorithms 31
6.1 The formulas side by side 31
6.2 Five questions to ask when you see a decision-tree problem 31

7 How to implement the algorithms yourself 32
7.1 The six building blocks 32
7.2 How to write a prediction function 32
7.3 A study sequence that builds understanding 32
7.4 A code-reconstruction drill 33
7.5 Questions to ask yourself while debugging 33

8 Additional practice problems 33

9 Answer checks for the additional practice 34

10 Final examination checklist 35

1 How to use these notes

The phrase *decision tree* describes a family of models, not one single algorithm. ID3, C4.5 and CART all build a tree by repeatedly dividing training data, but they do not use the same split rule or the same tree structure. Learn each algorithm separately before comparing them.

Key idea

The one-line distinction

- **ID3:** chooses the categorical attribute with the largest information gain, using entropy.
- **C4.5:** improves on ID3 by using gain ratio, handling continuous attributes through thresholds, and adding practical mechanisms such as missing-value handling and pruning.
- **CART:** builds binary splits. For classification it commonly uses Gini impurity; for regression it minimizes squared error / variance.

1.1 The fixed format used for every solved problem

Every numerical problem in this document follows the same sequence. Use the same structure in written examinations so that the marker can follow the logic.

1. **Problem and objective:** state what must be selected or calculated.
2. **Given data:** write the parent class counts and the child-group counts.
3. **Formula:** write the exact impurity or split formula before substituting values.
4. **Parent calculation:** calculate the impurity before splitting.
5. **Candidate calculation:** calculate each child impurity, its weighted value, and the split score.
6. **Comparison:** compare candidates using the algorithm's own criterion.
7. **Decision:** state the winning feature or threshold and why it wins.
8. **Tree update:** show what data reaches each child and what happens next.

1.2 What you should be able to do after studying

- Explain why a tree is a recursive partition of the feature space.
- Calculate entropy, information gain, split information, gain ratio and Gini impurity by hand.
- Find a threshold for a continuous feature by sorting values and testing candidate midpoints.
- Draw multiway and binary trees correctly.
- Distinguish classification trees from regression trees.
- Write a small training and prediction implementation without scikit-learn.

2 Shared mathematical foundations

This chapter contains the mathematics that will be needed by the algorithms. It does not yet prescribe which algorithm uses which formula.

2.1 Supervised learning setup

Assume a labelled training dataset

$$D = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \ldots, (\mathbf{x}_N, y_N)\},$$

where $\mathbf{x}_i = (x_{i1}, x_{i2}, \ldots, x_{im})$ contains the features of example i, and y_i is its target.

For classification, the target belongs to one of K classes:

$$y_i \in \{C_1, C_2, \ldots, C_K\}.$$

For regression, the target is numeric:

$$y_i \in \mathbb{R}.$$

A tree node represents a subset $S \subseteq D$. The root contains all training examples. A split sends examples into child subsets. Repeating this process creates a tree.

2.2 What a decision tree learns

A decision tree learns a sequence of tests. A classification tree may contain a rule such as:

$$\text{Outlook} = \text{Sunny?}$$

A numeric feature may be tested as:

$$\text{Age} \leq 27.5?$$

At an internal node, the test decides which child receives an observation. A leaf stores a prediction: usually a class label (or class probabilities) for classification, and usually a mean target value for squared-error regression.

Key idea

A split is useful when its child groups are more homogeneous with respect to the target than the parent group. The algorithms differ mainly in how they measure that homogeneity and how they are allowed to split a feature.

2.3 Class proportions

Suppose a node contains N observations and n_k of them belong to class C_k. The class proportion is

$$p_k = \frac{n_k}{N}, \qquad \sum_{k=1}^{K} p_k = 1.$$

For a binary target with n_+ positives and n_- negatives:

$$p_+ = \frac{n_+}{N}, \qquad p_- = \frac{n_-}{N}.$$

A node is **pure** if all observations belong to one class. In that case one class proportion equals 1 and all others equal 0.

2.4 Entropy: uncertainty in a class distribution

Entropy measures the uncertainty of a class label. For a node with class proportions $p_1, \ldots, p_K$,

$$\boxed{\mathrm{H}(S) = -\sum_{k=1}^{K} p_k \log_2 p_k}$$

with the convention $0 \log_2 0 = 0$.

For a binary class distribution,

$$\mathrm{H}(S) = -p_+ \log_2 p_+ - p_- \log_2 p_-.$$

Important properties:

- Entropy is 0 for a pure node.
- For two classes, entropy is highest at a 50:50 split and equals 1 bit.
- Entropy is not a model accuracy score. It measures class-mix uncertainty at a node.
- The logarithm base 2 gives entropy in bits. A different base changes the scale, not the ranking of candidate splits when used consistently.

Binary class proportions	Entropy (bits)
1.00 / 0.00	0.0000
0.90 / 0.10	0.4690
0.75 / 0.25	0.8113
0.50 / 0.50	1.0000

2.5 Weighted impurity after a split

Suppose a split divides parent node S into non-empty children $S_1, S_2, \ldots, S_r$. The weight of child j is its fraction of the parent observations:

$$w_j = \frac{|S_j|}{|S|}, \qquad \sum_{j=1}^{r} w_j = 1.$$

The weighted child entropy is

$$\boxed{\mathrm{H}_{\text{after}} = \sum_{j=1}^{r} \frac{|S_j|}{|S|}\,\mathrm{H}(S_j)}.$$

The same weighted-average structure is used with other impurity measures, such as Gini impurity.

Why weight the children? A child containing 90 observations should influence the parent score more than a child containing 2 observations. Taking an unweighted average would treat these children as equally important and would not measure the average uncertainty of a randomly selected training example.

2.6 Information gain

Information gain measures the reduction in entropy caused by a split:

$$\boxed{\mathrm{Gain}(S, A) = \mathrm{H}(S) - \sum_{v \in \mathrm{Values}(A)} \frac{|S_v|}{|S|}\,\mathrm{H}(S_v)}.$$

Here S_v is the subset for which attribute A takes value v.

A larger information gain means a larger reduction in entropy. Information gain is measured in bits when entropy uses $\log_2$.

2.7 Split information and gain ratio

The split information measures how broadly an attribute divides the observations, without considering class labels:

$$\boxed{\mathrm{SplitInfo}(S, A) = -\sum_{v} \frac{|S_v|}{|S|} \log_2\left(\frac{|S_v|}{|S|}\right)}.$$

Gain ratio normalizes information gain by split information:

$$\boxed{\mathrm{GainRatio}(S, A) = \frac{\mathrm{Gain}(S, A)}{\mathrm{SplitInfo}(S, A)}}.$$

If the split information is zero, the attribute did not actually divide the examples; its gain ratio is treated as undefined or zero and it is not selected.

2.8 Gini impurity

Gini impurity measures the chance of incorrectly classifying an observation if its label were assigned randomly according to the class proportions at the node:

$$\boxed{\text{Gini}(S) = 1 - \sum_{k=1}^{K} p_k^2}.$$

For a binary node:

$$\text{Gini}(S) = 1 - p_+^2 - p_-^2 = 2p_+p_-.$$

Properties:

- Gini is 0 for a pure node.
- For two classes it is largest at 50:50, where it equals 0.5.
- Like entropy, it is an impurity measure; unlike entropy, it does not use logarithms.

For a candidate split, the weighted Gini impurity is

$$\text{Gini}_{\text{after}} = \sum_{j=1}^{r} \frac{|S_j|}{|S|} \text{Gini}(S_j),$$

and the Gini reduction is

$$\boxed{\Delta\,\text{Gini} = \text{Gini}(S) - \text{Gini}_{\text{after}}\,.}$$

2.9 Regression impurity: squared error and variance

For a regression node containing target values $y_1, \ldots, y_N$, the prediction that minimizes squared error is the mean:

$$\bar{y} = \frac{1}{N} \sum_{i=1}^{N} y_i.$$

The sum of squared errors at that node is

$$\boxed{\text{SSE}(S) = \sum_{i=1}^{N} (y_i - \bar{y})^2.}$$

The population variance is $\text{SSE}(S)/N$. Since the parent size is fixed while comparing candidate splits at one node, minimizing the total child SSE is equivalent to minimizing the child-size-weighted variance.

For children $S_1, \ldots, S_r$,

$$\text{SSE}_{\text{after}} = \sum_{j=1}^{r} \sum_{i \in S_j} (y_i - \bar{y}_j)^2,$$

where $\bar{y}_j$ is the mean target in child j. A useful split has a large reduction:

$$\Delta\,\text{SSE} = \text{SSE}(S) - \text{SSE}_{\text{after}}\,.$$

2.10 Continuous features and candidate thresholds

A categorical feature may take values such as $\{\text{Sunny}, \text{Rain}, \text{Overcast}\}$. A numeric feature can take many distinct values, so a tree commonly creates a threshold test:

$$X \leq t \quad \text{versus} \quad X > t.$$

A standard educational method for finding candidate thresholds is:

1. Sort the training observations by the numeric feature.
2. List the distinct observed values $v_1 < v_2 < \cdots < v_q$.
3. Consider the midpoints between adjacent distinct values:

$$t_i = \frac{v_i + v_{i+1}}{2}, \quad i = 1, \ldots, q-1.$$

4. For each threshold, divide the rows into $X \leq t_i$ and $X > t_i$.
5. Calculate the algorithm-specific score and choose the best candidate.

The midpoint ensures that no training observation with value v_i is split ambiguously. Practical implementations may reduce the candidate set, for example by considering boundaries where the class changes.

2.11 Greedy recursive splitting

A decision-tree learner is usually greedy. At the current node it selects the best split according to its criterion, commits to that split, and then repeats the process independently inside each child. It does not normally search every possible complete tree in order to find the globally optimal tree.

2.12 Stopping, leaf predictions and overfitting

A tree can keep splitting until leaves contain one observation each. Such a tree may memorize the training data and generalize poorly. Common stopping conditions include:

- all observations at a node have the same class;
- no candidate split can produce a positive impurity reduction;
- the node has fewer than a minimum number of observations;
- a maximum depth has been reached;
- the improvement from the best split is too small.

When a branch cannot be split further, a classification leaf usually predicts the majority class in that node. A regression leaf using squared error predicts the mean target value.

Common mistake to avoid

A stopping rule is not the same thing as pruning. Stopping prevents a split before it is made (pre-pruning). Pruning removes or collapses parts of a tree after a larger tree has been grown (post-pruning). The distinction matters especially when discussing C4.5 and CART.

3 ID3: Iterative Dichotomiser 3

3.1 Purpose and defining idea

ID3 is a top-down, greedy decision-tree algorithm for classification. At each node it selects the attribute with the largest information gain, where information gain is calculated from entropy.

Key idea

ID3 split rule: choose the attribute A^* that maximizes information gain:

$$A^* = \arg\max_{A \in \mathcal{A}} \text{Gain}(S, A).$$

The original, basic ID3 formulation is designed for categorical attributes and produces a multiway branch for each attribute value.

3.2 How ID3 handles an attribute

Suppose the chosen attribute is Outlook and its observed values are Sunny, Overcast and Rain. ID3 makes three branches:

$$\text{Outlook} = \text{Sunny}, \quad \text{Outlook} = \text{Overcast}, \quad \text{Outlook} = \text{Rain}.$$

It does not normally turn this categorical attribute into a binary split. The algorithm then applies the same selection rule to the data subset at each child.

3.3 ID3 algorithm, step by step

1. Put all training examples at the root.
2. If every example has the same class, create a leaf with that class.
3. If no attributes remain, create a leaf with the majority class.
4. Calculate the parent entropy.
5. For each remaining categorical attribute, calculate the weighted child entropy and information gain.
6. Select the attribute with the largest information gain.
7. Create one child for every observed value of that attribute.
8. Remove the selected attribute from the available attribute list on that path.
9. Repeat the process for each child subset.

3.4 Pseudocode

```
ID3(examples, attributes):
    if all examples have the same class:
        return leaf(that class)

    if attributes is empty:
        return leaf(majority class in examples)

    A = attribute with maximum information gain
    node = decision_node(A)

    for each value v of A:
        subset = examples where A == v
        if subset is empty:
```

```
            child = leaf(majority class in examples)
        else:
            child = ID3(subset, attributes minus A)
        attach child to node using branch A == v

    return node
```

3.5 Solved numerical problem 1: selecting the ID3 root

Worked numerical problem

Problem. A training set has 14 observations: 9 labelled Yes and 5 labelled No. Four categorical attributes are available: Outlook, Temperature, Humidity and Wind. Select the root attribute using ID3.

Step 1 - Given parent counts.

$$N = 14, \qquad n_{Yes} = 9, \qquad n_{No} = 5.$$

Step 2 - Parent entropy.

$$\begin{aligned} \mathrm{H}(S) &= -\frac{9}{14}\log_2\frac{9}{14} - \frac{5}{14}\log_2\frac{5}{14} \\ &\approx 0.9403 \text{ bits.} \end{aligned}$$

Step 3 - Counts for each Outlook value.

Outlook value	*Yes*	*No*	Total
Sunny	2	3	5
Overcast	4	0	4
Rain	3	2	5
Total	9	5	14

Step 4 - Entropy of each child.

$$\mathrm{H}(Sunny) = -\frac{2}{5}\log_2\frac{2}{5} - \frac{3}{5}\log_2\frac{3}{5} \approx 0.9710.$$

$$\mathrm{H}(Overcast) = 0 \quad \text{(all four are Yes).}$$

$$\mathrm{H}(Rain) = -\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} \approx 0.9710.$$

Step 5 - Weighted entropy after the Outlook split.

$$\begin{aligned} \mathrm{H}_{after}(Outlook) &= \frac{5}{14}(0.9710) + \frac{4}{14}(0) + \frac{5}{14}(0.9710) \\ &\approx 0.6935. \end{aligned}$$

Step 6 - Information gain for Outlook.

$$\mathrm{Gain}(S, Outlook) = 0.9403 - 0.6935 = 0.2467.$$

Step 7 - Repeat for the remaining attributes. The class counts and resulting calculations are:

Attribute	Child class counts	H_{after}	Gain
Outlook	$(2,3), (4,0), (3,2)$	0.6935	0.2467
Humidity	$(3,4), (6,1)$	0.7885	0.1518
Wind	$(6,2), (3,3)$	0.8922	0.0481
Temperature	$(2,2), (4,2), (3,1)$	0.9111	0.0292

Each pair is written as (Yes count, No count). For example, Humidity has a High group with (3,4) and a Normal group with (6,1).

Step 8 - Compare gains.

$$0.2467 > 0.1518 > 0.0481 > 0.0292.$$

Final decision. ID3 selects **Outlook** as the root because it gives the largest information gain.

3.6 Solved numerical problem 2: completing the ID3 tree

Continue from the root selected above. The root creates three child subsets.

Sunny branch. Its class counts are 2 Yes and 3 No. The remaining attributes are Temperature, Humidity and Wind. Humidity divides the five rows as follows:

Humidity	*Yes*	*No*	Entropy
High	0	3	0
Normal	2	0	0

Both children are pure, so the Sunny branch splits on Humidity and becomes two leaves.

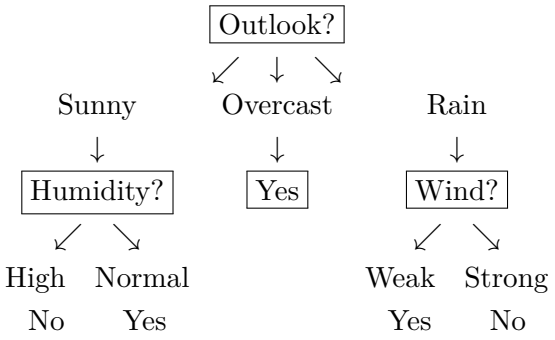
Overcast branch. Its four rows are all Yes. It immediately becomes a Yes leaf; no further calculation is needed.

Rain branch. Its class counts are 3 Yes and 2 No. Wind divides these rows into:

Wind	*Yes*	*No*	Entropy
Weak	3	0	0
Strong	0	2	0

Both children are pure, so the Rain branch splits on Wind.

Resulting tree.



3.7 What ID3 does well and where it is limited

Strengths:

- The split rule is mathematically clear and easy to calculate by hand.
- The resulting multiway tree is interpretable.
- It works naturally with categorical features.

Limitations of basic ID3:

- Information gain can favour attributes with many distinct values. An attribute that gives every row its own branch can produce pure leaves, even when that split is not useful for generalization.
- Basic ID3 does not natively define a threshold search for continuous-valued attributes.
- The original basic form does not provide the later pruning and missing-value machinery associated with C4.5.
- Without a sensible stopping rule, a tree can overfit.

3.8 From-scratch ID3 implementation

The following implementation uses only Python's standard language features. It assumes that every feature is categorical and every row is a dictionary. It intentionally does not add numeric thresholds,

missing-value weighting or post-pruning; those belong to later algorithm variants.

```python
from math import log2


def entropy(rows, target):
    """Entropy of the target labels in a list of row dictionaries."""
    counts = {}
    for row in rows:
        label = row[target]
        counts[label] = counts.get(label, 0) + 1

    total = len(rows)
    result = 0.0
    for count in counts.values():
        p = count / total
        result -= p * log2(p)
    return result


def majority_class(rows, target):
    """Return the most frequent class in rows."""
    counts = {}
    for row in rows:
        label = row[target]
        counts[label] = counts.get(label, 0) + 1
    return max(counts, key=counts.get)


def information_gain(rows, feature, target):
    """Information gain from a multiway categorical split."""
    parent_entropy = entropy(rows, target)

    groups = {}
    for row in rows:
        value = row[feature]
        groups.setdefault(value, []).append(row)

    weighted_child_entropy = 0.0
    for subset in groups.values():
        weight = len(subset) / len(rows)
        weighted_child_entropy += weight * entropy(subset, target)

    return parent_entropy - weighted_child_entropy


def id3(rows, features, target):
    """Build an ID3 tree. Leaves use {'kind': 'leaf', 'label': ...}."""
    # A pure node needs no more tests.
    labels = {row[target] for row in rows}
    if len(labels) == 1:
        return {"kind": "leaf", "label": next(iter(labels))}

    # If no feature remains, predict the local majority class.
    fallback = majority_class(rows, target)
    if not features:
        return {"kind": "leaf", "label": fallback}

    # Select the feature with the largest information gain.
    scores = [(information_gain(rows, f, target), f) for f in features]
    best_gain, best_feature = max(scores)
```

```
    # Do not create a split that makes no progress.
    if best_gain <= 1e-12:
        return {"kind": "leaf", "label": fallback}

    node = {
        "kind": "categorical",
        "feature": best_feature,
        "default": fallback,
        "children": {}
    }

    groups = {}
    for row in rows:
        value = row[best_feature]
        groups.setdefault(value, []).append(row)

    remaining = [f for f in features if f != best_feature]
    for value, subset in groups.items():
        node["children"][value] = id3(subset, remaining, target)

    return node


def predict_id3(tree, row):
    """Predict one row; unseen feature values use the node majority."""
    if tree["kind"] == "leaf":
        return tree["label"]

    value = row.get(tree["feature"])
    child = tree["children"].get(value)
    if child is None:
        return tree["default"]
    return predict_id3(child, row)
```

3.8.1 Small runnable example: Play Tennis

The next block supplies the rows needed to train and test the preceding ID3 functions. The target column is `Play`; all input features are categorical.

```
play_tennis = [
    {"Outlook": "Sunny",    "Temperature": "Hot",  "Humidity": "High",   "Wind": "Weak",
    "Play": "No"},
    {"Outlook": "Sunny",    "Temperature": "Hot",  "Humidity": "High",   "Wind": "Strong",
    "Play": "No"},
    {"Outlook": "Overcast", "Temperature": "Hot",  "Humidity": "High",   "Wind": "Weak",
    "Play": "Yes"},
    {"Outlook": "Rain",     "Temperature": "Mild", "Humidity": "High",   "Wind": "Weak",
    "Play": "Yes"},
    {"Outlook": "Rain",     "Temperature": "Cool", "Humidity": "Normal", "Wind": "Weak",
    "Play": "Yes"},
    {"Outlook": "Rain",     "Temperature": "Cool", "Humidity": "Normal", "Wind": "Strong",
    "Play": "No"},
    {"Outlook": "Overcast", "Temperature": "Cool", "Humidity": "Normal", "Wind": "Strong",
    "Play": "Yes"},
    {"Outlook": "Sunny",    "Temperature": "Mild", "Humidity": "High",   "Wind": "Weak",
    "Play": "No"},
    {"Outlook": "Sunny",    "Temperature": "Cool", "Humidity": "Normal", "Wind": "Weak",
    "Play": "Yes"},
```

```
    {"Outlook": "Rain",     "Temperature": "Mild", "Humidity": "Normal", "Wind": "Weak",
    "Play": "Yes"},
    {"Outlook": "Sunny",    "Temperature": "Mild", "Humidity": "Normal", "Wind": "Strong",
    "Play": "Yes"},
    {"Outlook": "Overcast", "Temperature": "Mild", "Humidity": "High",   "Wind": "Strong",
    "Play": "Yes"},
    {"Outlook": "Overcast", "Temperature": "Hot",  "Humidity": "Normal", "Wind": "Weak",
    "Play": "Yes"},
    {"Outlook": "Rain",     "Temperature": "Mild", "Humidity": "High",   "Wind": "Strong",
    "Play": "No"}
]

features = ["Outlook", "Temperature", "Humidity", "Wind"]
tree = id3(play_tennis, features, target="Play")

new_day = {
    "Outlook": "Sunny",
    "Temperature": "Cool",
    "Humidity": "High",
    "Wind": "Strong"
}
print(predict_id3(tree, new_day))
```

The prediction is `No`: the root sends Sunny observations to the Humidity test, and High leads to the No leaf.

3.8.2 How to read the ID3 code

1. `entropy` counts labels, converts counts to probabilities and applies the entropy formula.
2. `information_gain` first calculates the parent entropy, groups rows by feature value, then subtracts the weighted child entropy.
3. `id3` checks stopping conditions before selecting a feature.
4. The selected feature is removed from the list passed to each child, because this basic categorical ID3 version uses an attribute at most once along a path.
5. `predict_id3` follows the matching branch until it reaches a leaf.

4 C4.5: the gain-ratio decision tree

4.1 Purpose and relationship to ID3

C4.5 is a development of the ID3 family. It retains the top-down greedy tree-building idea, but addresses important practical limitations of basic ID3. In particular, it uses gain ratio to reduce the preference for attributes that create many branches, supports continuous attributes by selecting thresholds, and includes mechanisms for handling missing values and pruning.

Key idea

C4.5 split idea: use information gain and split information to calculate gain ratio. A practical C4.5 selection procedure also guards against choosing a very low-gain split merely because its split information is small.

4.2 Gain ratio: why normalize information gain?

Information gain can be biased towards attributes with many possible values. Imagine an identifier column in which almost every row has a different value. Splitting on that identifier can create many tiny or single-row branches, producing very pure children. Its information gain can therefore be large even though the attribute is not a meaningful predictor.

Gain ratio divides information gain by split information:

$$\text{GainRatio}(S, A) = \frac{\text{Gain}(S, A)}{\text{SplitInfo}(S, A)}.$$

The denominator measures how widely the data are fragmented by the split. It penalizes a split that creates many or highly unbalanced partitions.

Important nuance: gain ratio itself can sometimes favour a split with a very small information gain if its split-information denominator is also small. The C4.5 selection procedure therefore uses an average-information-gain safeguard: among candidate attributes, it gives preference to those whose information gain is at least the average gain, and then selects the eligible candidate with the largest gain ratio. Exact implementation details can vary between textbook presentations and software versions; the educational implementation below makes this safeguard explicit.

4.3 Categorical versus continuous attributes in C4.5

Categorical feature. A categorical attribute is typically split into one child per value, as in the ID3 family.

Continuous feature. C4.5 searches for a threshold. For a candidate threshold t, it creates two groups:

$$S_{\leq t} = \{x \in S : X \leq t\}, \qquad S_{>t} = \{x \in S : X > t\}.$$

It calculates the information gain, split information and gain ratio for these two groups. A continuous feature can be used again farther down the tree with a different threshold; it is not normally removed after one numeric test.

4.4 Missing values and pruning: conceptual treatment

Missing values. Classic C4.5 can use examples with known values to estimate a split, and can distribute the influence of examples with unknown values across branches using branch probabilities. A simplified teaching implementation may instead reject or omit missing-value handling. Do not assume every implementation of a C4.5-like algorithm uses exactly the same missing-value convention.

Pruning. C4.5 uses post-pruning ideas to reduce branches that do not justify their complexity. A larger tree is grown, then parts of it may be replaced by a leaf or subtree when doing so improves estimated generalization performance. This is conceptually different from merely stopping every branch as soon as a local rule says not to split.

4.5 Solved numerical problem 3: calculating gain ratio on the Play Tennis data

Worked numerical problem

Problem. Using the same 14 observations from the ID3 example, calculate the gain ratio for Outlook, Humidity, Wind and Temperature. Determine which attribute is selected by the educational C4.5 rule.

Step 1 - Parent entropy.

$$\mathrm{H}(S) = 0.9403.$$

Step 2 - Recall the information gains.

Attribute	Gain(S, A)
Outlook	0.2467
Humidity	0.1518
Wind	0.0481
Temperature	0.0292

Step 3 - Calculate split information. For Outlook, the branch sizes are 5, 4 and 5:

$$\begin{aligned}\mathrm{SplitInfo}(S, \mathit{Outlook}) &= -\frac{5}{14}\log_2\frac{5}{14} - \frac{4}{14}\log_2\frac{4}{14} - \frac{5}{14}\log_2\frac{5}{14} \\ &\approx 1.5774.\end{aligned}$$

For Humidity, the branch sizes are 7 and 7, so SplitInfo $= 1.0000$. For Wind, the sizes are 8 and 6, so SplitInfo ≈ 0.9852. For Temperature, the sizes are 4, 6 and 4, so SplitInfo ≈ 1.5567.

Step 4 - Divide gain by split information.

Attribute	Gain	SplitInfo	Gain ratio
Outlook	0.2467	1.5774	0.1564
Humidity	0.1518	1.0000	0.1518
Wind	0.0481	0.9852	0.0488
Temperature	0.0292	1.5567	0.0188

Step 5 - Apply the average-gain safeguard.

$$\overline{\mathit{Gain}} = \frac{0.2467 + 0.1518 + 0.0481 + 0.0292}{4} \approx 0.1190.$$

Outlook and Humidity have gains at least as large as the average. Their gain ratios are 0.1564 and 0.1518, respectively.

Final decision. Outlook has the larger eligible gain ratio, so the educational C4.5 selection rule chooses **Outlook**.

4.6 Solved numerical problem 4: choosing a numeric threshold in C4.5

Worked numerical problem

Problem. A numeric feature Score and binary target are observed as follows. Select the best threshold using gain ratio.

Score	2	3	4	5	6	7	8	9
Class	*No*	*No*	*Yes*	*Yes*	*Yes*	*No*	*No*	*No*

Step 1 - Sort the observations. They are already sorted. Candidate midpoints between adjacent distinct values are 2.5, 3.5, 4.5, 5.5, 6.5, 7.5 and 8.5. The most informative candidates are evaluated using the same formula; below we compare 3.5 and 6.5 in detail.
Step 2 - Parent entropy. There are 3 Yes and 5 No observations:

$$\mathrm{H}(S) = -\frac{3}{8}\log_2\frac{3}{8} - \frac{5}{8}\log_2\frac{5}{8} \approx 0.9544.$$

Step 3 - Candidate threshold $t = 3.5$. Left child (Score ≤ 3.5): 0 Yes, 2 No, entropy 0. Right child (Score > 3.5): 3 Yes, 3 No, entropy 1.

$$\mathrm{H}_{after} = \frac{2}{8}(0) + \frac{6}{8}(1) = 0.7500.$$

$$\mathrm{Gain} = 0.9544 - 0.7500 = 0.2044.$$

$$\mathrm{SplitInfo} = -\frac{2}{8}\log_2\frac{2}{8} - \frac{6}{8}\log_2\frac{6}{8} = 0.8113.$$

$$\mathrm{GainRatio} = \frac{0.2044}{0.8113} \approx 0.2520.$$

Step 4 - Candidate threshold $t = 6.5$. Left child: 3 Yes, 2 No, entropy

$$-\frac{3}{5}\log_2\frac{3}{5} - \frac{2}{5}\log_2\frac{2}{5} \approx 0.9710.$$

Right child: 0 Yes, 3 No, entropy 0.

$$\mathrm{H}_{after} = \frac{5}{8}(0.9710) + \frac{3}{8}(0) = 0.6068.$$

$$\mathrm{Gain} = 0.9544 - 0.6068 = 0.3476.$$

$$\mathrm{SplitInfo} = -\frac{5}{8}\log_2\frac{5}{8} - \frac{3}{8}\log_2\frac{3}{8} = 0.9544.$$

$$\mathrm{GainRatio} = \frac{0.3476}{0.9544} \approx 0.3642.$$

Step 5 - Compare.

$$0.3642 > 0.2520.$$

The threshold 6.5 has the larger gain ratio among these candidates. In this dataset it also gives a strong, clean first split: all three observations above 6.5 are No.
Final decision. Select $\boxed{Score \leq 6.5 \text{ versus } Score > 6.5}$. Recursively inspect the left child because it still contains both classes.

4.7 From-scratch educational C4.5 implementation

This implementation supports:

- categorical features with one branch per observed value;

- numeric features with binary threshold tests;
- information gain, split information and gain ratio;
- the average-gain eligibility safeguard described above;
- majority-class fallback for an unseen categorical value.

It assumes complete data (no missing values), scalar categorical values, and a classification target. It is a transparent teaching implementation, not a claim to reproduce every optimization or missing-value detail in a production C4.5 implementation.

```python
from math import log2


def class_entropy(rows, target):
    counts = {}
    for row in rows:
        label = row[target]
        counts[label] = counts.get(label, 0) + 1

    total = len(rows)
    h = 0.0
    for count in counts.values():
        p = count / total
        h -= p * log2(p)
    return h


def majority(rows, target):
    counts = {}
    for row in rows:
        label = row[target]
        counts[label] = counts.get(label, 0) + 1
    return max(counts, key=counts.get)


def score_partition(rows, parts, target):
    """Return gain, split information and gain ratio for child row lists."""
    parent_h = class_entropy(rows, target)
    weighted_h = 0.0
    split_info = 0.0
    total = len(rows)

    for subset in parts.values():
        weight = len(subset) / total
        weighted_h += weight * class_entropy(subset, target)
        split_info -= weight * log2(weight)

    gain = parent_h - weighted_h
    ratio = gain / split_info if split_info > 0 else 0.0
    return gain, split_info, ratio


def candidates_for_feature(rows, feature, target, numeric_features):
    """Generate the candidate partitions for one feature."""
    candidates = []

    if feature in numeric_features:
        values = sorted({row[feature] for row in rows})
        for i in range(len(values) - 1):
```

```
            threshold = (values[i] + values[i + 1]) / 2
            left = [r for r in rows if r[feature] <= threshold]
            right = [r for r in rows if r[feature] > threshold]
            if not left or not right:
                continue
            parts = {"left": left, "right": right}
            gain, split_info, ratio = score_partition(rows, parts, target)
            candidates.append({
                "feature": feature,
                "kind": "numeric",
                "threshold": threshold,
                "parts": parts,
                "gain": gain,
                "split_info": split_info,
                "ratio": ratio
            })
    else:
        groups = {}
        for row in rows:
            groups.setdefault(row[feature], []).append(row)
        if len(groups) > 1:
            gain, split_info, ratio = score_partition(rows, groups, target)
            candidates.append({
                "feature": feature,
                "kind": "categorical",
                "parts": groups,
                "gain": gain,
                "split_info": split_info,
                "ratio": ratio
            })

    return candidates


def best_gain_candidate(rows, feature, target, numeric_features):
    options = candidates_for_feature(rows, feature, target, numeric_features)
    if not options:
        return None
    # For a numeric feature, first retain its threshold with the highest gain.
    return max(options, key=lambda item: item["gain"])


def c45(rows, features, numeric_features, target):
    """Build an educational C4.5-style classification tree."""
    labels = {row[target] for row in rows}
    if len(labels) == 1:
        return {"kind": "leaf", "label": next(iter(labels))}

    fallback = majority(rows, target)
    if not features:
        return {"kind": "leaf", "label": fallback}

    # Find the best-gain candidate for each available feature.
    candidates = []
    for feature in features:
        candidate = best_gain_candidate(
            rows, feature, target, numeric_features
        )
        if candidate is not None:
            candidates.append(candidate)
```

```
    if not candidates:
        return {"kind": "leaf", "label": fallback}

    average_gain = sum(c["gain"] for c in candidates) / len(candidates)
    eligible = [c for c in candidates if c["gain"] >= average_gain]
    if not eligible:
        return {"kind": "leaf", "label": fallback}

    best = max(eligible, key=lambda item: item["ratio"])
    if best["gain"] <= 1e-12:
        return {"kind": "leaf", "label": fallback}

    feature = best["feature"]
    node = {
        "kind": best["kind"],
        "feature": feature,
        "default": fallback
    }

    if best["kind"] == "numeric":
        node["threshold"] = best["threshold"]
        node["left"] = c45(
            best["parts"]["left"], features, numeric_features, target
        )
        node["right"] = c45(
            best["parts"]["right"], features, numeric_features, target
        )
    else:
        node["children"] = {}
        remaining = [f for f in features if f != feature]
        for value, subset in best["parts"].items():
            node["children"][value] = c45(
                subset, remaining, numeric_features, target
            )

    return node


def predict_c45(tree, row):
    if tree["kind"] == "leaf":
        return tree["label"]

    feature = tree["feature"]
    if tree["kind"] == "numeric":
        if row[feature] <= tree["threshold"]:
            return predict_c45(tree["left"], row)
        return predict_c45(tree["right"], row)

    value = row.get(feature)
    child = tree["children"].get(value)
    if child is None:
        return tree["default"]
    return predict_c45(child, row)
```

4.7.1 Small runnable example: a numeric feature

This example reuses the eight Score observations from the worked threshold problem. The numeric feature set tells the builder which column needs threshold candidates.

```
score_rows = [
    {"Score": 2, "Class": "No"},
    {"Score": 3, "Class": "No"},
    {"Score": 4, "Class": "Yes"},
    {"Score": 5, "Class": "Yes"},
    {"Score": 6, "Class": "Yes"},
    {"Score": 7, "Class": "No"},
    {"Score": 8, "Class": "No"},
    {"Score": 9, "Class": "No"}
]

tree = c45(
    score_rows,
    features=["Score"],
    numeric_features={"Score"},
    target="Class"
)
print(predict_c45(tree, {"Score": 5.5}))
```

The example predicts `Yes`. The first threshold selected is 6.5; the left child then receives a second threshold test that isolates the two No observations at Scores 2 and 3 from the three Yes observations at Scores 4, 5 and 6.

4.7.2 How to trace the C4.5 code

1. `candidates_for_feature` creates partitions. Numeric values produce threshold candidates; categorical values produce a multiway partition.
2. `score_partition` calculates three quantities separately: gain, split information and gain ratio.
3. `best_gain_candidate` retains the threshold with the largest raw gain for each numeric feature.
4. `c45` computes the average of the per-feature gains, filters candidates, and selects the eligible candidate with the highest ratio.
5. Numeric features remain available in both descendants; a different threshold can be selected deeper in the tree. A categorical feature is removed after its multiway split.

5 CART: Classification and Regression Trees

5.1 Purpose and defining structure

CART stands for **Classification and Regression Trees**. It is one framework for both classification and regression. Its defining structural property is that every internal split is binary: each node sends observations to a left child or a right child.

Key idea

CART structure: every internal node makes exactly two-way decisions. A categorical split may group several categories on the left and the remaining categories on the right. A numeric split is usually $X \leq t$ versus $X > t$.

5.2 CART for classification

A common CART classification criterion is Gini impurity. For a node S,

$$\text{Gini}(S) = 1 - \sum_{k=1}^{K} p_k^2.$$

For every candidate binary split, CART calculates

$$\text{Gini}_{after} = \frac{|S_L|}{|S|}\,\text{Gini}(S_L) + \frac{|S_R|}{|S|}\,\text{Gini}(S_R).$$

It chooses the split with the smallest weighted child Gini, or equivalently the largest Gini reduction:

$$\Delta\,\text{Gini} = \text{Gini}(S) - \text{Gini}_{after}\,.$$

CART classification is not defined by gain ratio. Do not substitute the C4.5 formula into a CART calculation.

5.3 How CART creates binary splits

Numeric feature:

$$X \le t \quad \text{versus} \quad X > t.$$

Test candidate thresholds and select the one with the largest impurity reduction.

Categorical feature: if a feature has categories $\{A, B, C, D\}$, a binary split can be $\{A, C\}$ versus $\{B, D\}$. CART searches for a grouping of categories into two non-empty subsets. A simple implementation can enumerate category subsets; optimized implementations use more efficient methods in particular settings.

5.4 Solved numerical problem 5: selecting a categorical CART split using Gini

Worked numerical problem

Problem. The 14-row Play Tennis dataset contains 9 Yes and 5 No examples. At the root, consider three possible binary groupings of Outlook: Sunny versus the rest, Overcast versus the rest, and Rain versus the rest. Which split does CART select using Gini impurity?

Step 1 - Parent Gini.

$$\begin{aligned}\text{Gini}(S) &= 1 - \left(\frac{9}{14}\right)^2 - \left(\frac{5}{14}\right)^2 \\ &= 1 - \frac{81}{196} - \frac{25}{196} \\ &= \frac{90}{196} \approx 0.4592.\end{aligned}$$

Step 2 - Candidate A: Sunny versus non-Sunny. Sunny contains 2 Yes and 3 No:

$$\text{Gini}(Sunny) = 1 - \left(\frac{2}{5}\right)^2 - \left(\frac{3}{5}\right)^2 = 0.4800.$$

The other rows contain 7 Yes and 2 No:

$$\text{Gini}(NonSunny) = 1 - \left(\frac{7}{9}\right)^2 - \left(\frac{2}{9}\right)^2 = \frac{28}{81} \approx 0.3457.$$

Weighted child Gini:

$$\text{Gini}_{after} = \frac{5}{14}(0.4800) + \frac{9}{14}(0.3457) \approx 0.3937.$$

Reduction:

$$\Delta\,\text{Gini} = 0.4592 - 0.3937 = 0.0655.$$

Step 3 - Candidate B: Overcast versus non-Overcast. Overcast contains 4 Yes and 0 No, so its Gini is 0. The remaining 10 rows contain 5 Yes and 5 No, so their Gini is 0.5.

$$\text{Gini}_{after} = \frac{4}{14}(0) + \frac{10}{14}(0.5) = 0.3571.$$

$$\Delta\,\text{Gini} = 0.4592 - 0.3571 = 0.1020.$$

Step 4 - Candidate C: Rain versus non-Rain. Rain contains 3 Yes and 2 No, so its Gini is 0.48. The other 9 rows contain 6 Yes and 3 No, so their Gini is

$$1 - \left(\frac{6}{9}\right)^2 - \left(\frac{3}{9}\right)^2 = 0.4444.$$

$$\text{Gini}_{after} = \frac{5}{14}(0.48) + \frac{9}{14}(0.4444) \approx 0.4571.$$

$$\Delta\,\text{Gini} = 0.4592 - 0.4571 = 0.0020.$$

Step 5 - Compare.

Binary split	Gini_{after}	Δ Gini
Sunny vs rest	0.3937	0.0655
Overcast vs rest	0.3571	0.1020
Rain vs rest	0.4571	0.0020

Final decision. CART chooses **Overcast versus non-Overcast**, because it has the lowest weighted child Gini and the largest Gini reduction among the three candidates.

5.5 Solved numerical problem 6: numeric threshold using CART Gini

Worked numerical problem

Problem. Use the Score dataset from the C4.5 numerical problem. Select between thresholds 3.5 and 6.5 using CART's Gini criterion.

Step 1 - Parent Gini. There are 3 Yes and 5 No:

$$\text{Gini}(S) = 1 - \left(\frac{3}{8}\right)^2 - \left(\frac{5}{8}\right)^2 = 0.46875.$$

Step 2 - Threshold 3.5. Left: 0 Yes, 2 No, so Gini is 0. Right: 3 Yes, 3 No, so Gini is 0.5.

$$\text{Gini}_{after} = \frac{2}{8}(0) + \frac{6}{8}(0.5) = 0.3750,$$

$$\Delta\,\text{Gini} = 0.46875 - 0.3750 = 0.09375.$$

Step 3 - Threshold 6.5. Left: 3 Yes, 2 No, so Gini is

$$1 - \left(\frac{3}{5}\right)^2 - \left(\frac{2}{5}\right)^2 = 0.48.$$

Right: 0 Yes, 3 No, so Gini is 0.

$$\text{Gini}_{after} = \frac{5}{8}(0.48) + \frac{3}{8}(0) = 0.3000,$$

$$\Delta\,\text{Gini} = 0.46875 - 0.3000 = 0.16875.$$

Step 4 - Compare.

$$0.16875 > 0.09375.$$

Final decision. Select $\boxed{Score \leq 6.5 \text{ versus } Score > 6.5}$ because it gives the greater Gini reduction.

5.6 CART for regression

For regression, there are no class proportions. A leaf predicts the mean target value of the observations that reach it. The split is selected to reduce within-child squared error.

At a node S:

$$\bar{y}_S = \frac{1}{|S|}\sum_{i \in S} y_i, \qquad \text{SSE}(S) = \sum_{i \in S}(y_i - \bar{y}_S)^2.$$

For a candidate binary split:

$$\text{SSE}_{after} = \text{SSE}(S_L) + \text{SSE}(S_R).$$

Choose the split minimizing SSE_{after}, equivalently maximizing $\text{SSE}(S) - \text{SSE}_{after}$.

5.7 Solved numerical problem 7: a CART regression split

Worked numerical problem

Problem. A regression dataset has one ordered feature X and target Y:

X	1	2	3	4	5	6
Y	2	3	4	8	9	10

Find the best root threshold using squared error.
Step 1 - Parent mean and SSE.

$$\bar{y} = \frac{2+3+4+8+9+10}{6} = 6.$$

$$\begin{aligned}\mathrm{SSE}(S) &= (2-6)^2 + (3-6)^2 + (4-6)^2 \\ &\quad + (8-6)^2 + (9-6)^2 + (10-6)^2 \\ &= 16+9+4+4+9+16 = 58.\end{aligned}$$

Step 2 - Candidate split at $X \le 3.5$. Left targets: (2,3,4), mean $\bar{y}_L = 3$.

$$\mathrm{SSE}_L = (2-3)^2 + (3-3)^2 + (4-3)^2 = 2.$$

Right targets: (8,9,10), mean $\bar{y}_R = 9$.

$$\mathrm{SSE}_R = (8-9)^2 + (9-9)^2 + (10-9)^2 = 2.$$

Thus

$$\mathrm{SSE}_{after} = 2+2 = 4, \qquad \Delta\,\mathrm{SSE} = 58-4 = 54.$$

Step 3 - Compare another candidate, $X \le 2.5$. Left targets: (2,3), mean 2.5, giving $\mathrm{SSE}_L = 0.5$. Right targets: (4,8,9,10), mean 7.75:

$$\begin{aligned}\mathrm{SSE}_R &= (4-7.75)^2 + (8-7.75)^2 \\ &\quad + (9-7.75)^2 + (10-7.75)^2 \\ &= 14.0625 + 0.0625 + 1.5625 + 5.0625 = 20.75.\end{aligned}$$

So $\mathrm{SSE}_{after} = 21.25$, which is worse than 4. The symmetric threshold $X \le 4.5$ also gives 21.25. Testing all candidate midpoints identifies 3.5 as the minimum-SSE split.
Final decision. Choose $\boxed{X \le 3.5 \text{ versus } X > 3.5}$. The two leaves predict 3 and 9, respectively. The split reduces the node SSE from 58 to 4.

5.8 Cost-complexity pruning in CART

A very deep CART tree can fit noise. CART commonly uses cost-complexity pruning. The guiding objective is

$$\boxed{R_\alpha(T) = R(T) + \alpha|T|,}$$

where:

- T is a tree;
- $R(T)$ is the tree's training error or impurity-based risk (as defined for the task);
- $|T|$ is the number of terminal leaves;
- $\alpha \ge 0$ controls the penalty for tree size.

When $\alpha = 0$, complexity is not penalized by this term. Larger α values favour smaller trees. A practical workflow grows a large tree, generates a sequence of pruned subtrees, and uses validation or cross-validation to choose an appropriate complexity level.

Common mistake to avoid

Do not confuse the CART regression criterion with its pruning objective. The regression split criterion selects a local split by reducing squared error. Cost-complexity pruning is a separate model-complexity step applied to compare smaller subtrees.

5.9 From-scratch CART implementation: classification and regression

The following code is longer because CART supports both target types and permits binary grouping of categorical values. It uses a small-data exhaustive search for categorical groupings. This is easy to understand but can become expensive when a categorical feature has many categories.

```
from itertools import combinations


def gini(rows, target):
    counts = {}
    for row in rows:
        label = row[target]
        counts[label] = counts.get(label, 0) + 1

    total = len(rows)
    return 1.0 - sum((count / total) ** 2
                     for count in counts.values())


def cart_candidate_splits(rows, feature, numeric_features):
    """Yield binary partitions for one numeric or categorical feature."""
    if feature in numeric_features:
        values = sorted({row[feature] for row in rows})
        for i in range(len(values) - 1):
            threshold = (values[i] + values[i + 1]) / 2
            left = [r for r in rows if r[feature] <= threshold]
            right = [r for r in rows if r[feature] > threshold]
            if left and right:
                yield {
                    "kind": "numeric",
                    "threshold": threshold,
                    "left": left,
                    "right": right
                }
    else:
        values = sorted({row[feature] for row in rows}, key=str)
        if len(values) < 2:
            return

        # Fix the first category on the left to avoid mirror duplicates.
        first = values[0]
        remaining = values[1:]
        for size in range(len(remaining)):
            for chosen in combinations(remaining, size):
                left_values = set((first,) + chosen)
                if len(left_values) == len(values):
                    continue
                left = [r for r in rows if r[feature] in left_values]
                right = [r for r in rows if r[feature] not in left_values]
```

```
                if left and right:
                    yield {
                        "kind": "categorical",
                        "left_values": left_values,
                        "left": left,
                        "right": right
                    }


def majority_label(rows, target):
    counts = {}
    for row in rows:
        label = row[target]
        counts[label] = counts.get(label, 0) + 1
    return max(counts, key=counts.get)


def best_cart_classification_split(rows, features, numeric_features,
                                   target):
    best = None
    best_child_gini = float("inf")

    for feature in features:
        for split in cart_candidate_splits(rows, feature, numeric_features):
            left, right = split["left"], split["right"]
            weighted = (len(left) / len(rows)) * gini(left, target)
            weighted += (len(right) / len(rows)) * gini(right, target)

            if weighted < best_child_gini:
                best_child_gini = weighted
                best = dict(split)
                best["feature"] = feature
                best["child_gini"] = weighted

    return best


def cart_classification(rows, features, numeric_features, target,
                        depth=0, max_depth=10,
                        min_samples_split=2,
                        min_impurity_decrease=1e-12):
    labels = {row[target] for row in rows}
    prediction = majority_label(rows, target)

    if len(labels) == 1:
        return {"kind": "leaf", "label": prediction}
    if depth >= max_depth or len(rows) < min_samples_split:
        return {"kind": "leaf", "label": prediction}

    parent_impurity = gini(rows, target)
    split = best_cart_classification_split(
        rows, features, numeric_features, target
    )
    if split is None:
        return {"kind": "leaf", "label": prediction}

    decrease = parent_impurity - split["child_gini"]
    if decrease <= min_impurity_decrease:
        return {"kind": "leaf", "label": prediction}

```

```
    node = {
        "kind": split["kind"],
        "feature": split["feature"],
        "default": prediction
    }
    if split["kind"] == "numeric":
        node["threshold"] = split["threshold"]
    else:
        node["left_values"] = split["left_values"]

    node["left"] = cart_classification(
        split["left"], features, numeric_features, target,
        depth + 1, max_depth, min_samples_split, min_impurity_decrease
    )
    node["right"] = cart_classification(
        split["right"], features, numeric_features, target,
        depth + 1, max_depth, min_samples_split, min_impurity_decrease
    )
    return node


def predict_cart_classification(tree, row):
    if tree["kind"] == "leaf":
        return tree["label"]

    feature = tree["feature"]
    if tree["kind"] == "numeric":
        go_left = row[feature] <= tree["threshold"]
    else:
        go_left = row.get(feature) in tree["left_values"]

    child = tree["left"] if go_left else tree["right"]
    return predict_cart_classification(child, row)


# ---------- CART regression ----------

def mean(values):
    return sum(values) / len(values)


def sse(rows, target):
    values = [row[target] for row in rows]
    center = mean(values)
    return sum((value - center) ** 2 for value in values)


def best_cart_regression_split(rows, features, numeric_features, target):
    best = None
    best_child_sse = float("inf")

    for feature in features:
        for split in cart_candidate_splits(rows, feature, numeric_features):
            child_error = (sse(split["left"], target)
                           + sse(split["right"], target))
            if child_error < best_child_sse:
                best_child_sse = child_error
                best = dict(split)
                best["feature"] = feature
                best["child_sse"] = child_error
```

```
    return best


def cart_regression(rows, features, numeric_features, target,
                    depth=0, max_depth=10,
                    min_samples_split=2,
                    min_sse_decrease=1e-12):
    prediction = mean([row[target] for row in rows])
    if depth >= max_depth or len(rows) < min_samples_split:
        return {"kind": "leaf", "value": prediction}

    parent_error = sse(rows, target)
    split = best_cart_regression_split(
        rows, features, numeric_features, target
    )
    if split is None:
        return {"kind": "leaf", "value": prediction}

    decrease = parent_error - split["child_sse"]
    if decrease <= min_sse_decrease:
        return {"kind": "leaf", "value": prediction}

    node = {
        "kind": split["kind"],
        "feature": split["feature"]
    }
    if split["kind"] == "numeric":
        node["threshold"] = split["threshold"]
    else:
        node["left_values"] = split["left_values"]

    node["left"] = cart_regression(
        split["left"], features, numeric_features, target,
        depth + 1, max_depth, min_samples_split, min_sse_decrease
    )
    node["right"] = cart_regression(
        split["right"], features, numeric_features, target,
        depth + 1, max_depth, min_samples_split, min_sse_decrease
    )
    return node


def predict_cart_regression(tree, row):
    if tree["kind"] == "leaf":
        return tree["value"]

    feature = tree["feature"]
    if tree["kind"] == "numeric":
        go_left = row[feature] <= tree["threshold"]
    else:
        go_left = row.get(feature) in tree["left_values"]

    child = tree["left"] if go_left else tree["right"]
    return predict_cart_regression(child, row)
```

5.9.1 Small runnable examples

The first example trains a classification tree using a numeric feature. The second trains a regression tree; in both cases the training functions and prediction functions are the ones defined above.

```
score_rows = [
    {"Score": 2, "Class": "No"},
    {"Score": 3, "Class": "No"},
    {"Score": 4, "Class": "Yes"},
    {"Score": 5, "Class": "Yes"},
    {"Score": 6, "Class": "Yes"},
    {"Score": 7, "Class": "No"},
    {"Score": 8, "Class": "No"},
    {"Score": 9, "Class": "No"}
]

classification_tree = cart_classification(
    score_rows,
    features=["Score"],
    numeric_features={"Score"},
    target="Class",
    max_depth=5
)
print(predict_cart_classification(
    classification_tree, {"Score": 5.5}
))

regression_rows = [
    {"X": 1, "Y": 2}, {"X": 2, "Y": 3},
    {"X": 3, "Y": 4}, {"X": 4, "Y": 8},
    {"X": 5, "Y": 9}, {"X": 6, "Y": 10}
]

regression_tree = cart_regression(
    regression_rows,
    features=["X"],
    numeric_features={"X"},
    target="Y",
    max_depth=3
)
print(predict_cart_regression(regression_tree, {"X": 2.5}))
```

The classification example returns a class label; the regression example returns a numeric leaf mean. A CART tree remains binary in both tasks.

5.9.2 Implementation boundaries for this CART code

- It implements greedy binary splitting and simple pre-pruning controls (maximum depth, minimum node size and minimum impurity reduction).
- It does not implement the full cost-complexity pruning sequence. That concept is explained mathematically above.
- Categorical subset search is exhaustive and therefore suitable only for small numbers of distinct categories.
- Missing values are not handled. Prepare complete feature rows before calling the functions.
- The code uses the same split-generation routine for classification and regression, but the objective is different: weighted Gini for classification, total child SSE for regression.

6 A direct comparison: do not mix the algorithms

Question	ID3	C4.5	CART
Primary task	Classification	Classification	Classification and regression
Classification split criterion	Information gain (entropy reduction)	Gain ratio, with an information-gain safeguard in the usual selection procedure	Commonly Gini impurity reduction
Regression split criterion	Not part of basic ID3	Not part of the standard C4.5 classification algorithm	Squared-error / variance reduction
Categorical split shape	Usually one child per value	Usually one child per value	Exactly two groups per internal split
Numeric feature	Not native to basic ID3; discretize first if using the basic form	Searches for a threshold; a numeric feature may be reused deeper in the tree	Searches for a threshold and creates a binary split
Feature reuse	Basic categorical form removes the selected feature on that path	Categorical feature is usually removed after use; numeric feature can be reused with another threshold	Features can be reused in descendant nodes
Missing values	Basic form does not supply the C4.5-style mechanism	Has mechanisms for distributing missing-value influence	Handling depends on implementation; surrogate splits are used in some CART implementations
Pruning	Not a defining part of basic ID3	Includes post-pruning ideas	Cost-complexity pruning is a defining CART technique
Output structure	Multiway categorical tree	Multiway categorical and binary numeric tests	Binary tree only

6.1 The formulas side by side

Method	Selection quantity
ID3	$\text{Gain} = \text{H}(\textit{parent}) - \sum_j w_j \, \text{H}(\textit{child}_j)$; maximize it.
C4.5	$\text{GainRatio} = \dfrac{\text{Gain}}{\text{SplitInfo}}$; after the average-gain safeguard, maximize it.
CART classification	$\text{Gini}_{after} = \sum_j w_j \, \text{Gini}(\textit{child}_j)$; minimize it (or maximize Gini reduction).
CART regression	$\text{SSE}_{after} = \sum_j \text{SSE}(\textit{child}_j)$; minimize it (or maximize SSE reduction).

6.2 Five questions to ask when you see a decision-tree problem

1. Is the problem classification or regression?

2. Which algorithm has been requested?
3. Does the algorithm make multiway or binary splits?
4. Which exact impurity formula must be used?
5. Does the question give a categorical attribute, a numeric attribute, or both?

If the problem says ID3, do not calculate Gini. If it says C4.5, do not stop after information gain: calculate split information and gain ratio. If it says CART, do not use gain ratio: use the classification or regression criterion appropriate to the target.

7 How to implement the algorithms yourself

The goal is not to memorize a large block of code. The goal is to understand a small set of reusable operations and then assemble them in the correct order.

7.1 The six building blocks

1. **Count labels.** Given rows, count how many examples belong to each class.
2. **Measure a node.** Turn those counts into entropy or Gini. For regression, calculate the mean and SSE.
3. **Partition rows.** Given a feature and a rule, place each row into the correct child list.
4. **Score the partition.** Calculate the weighted child impurity and the relevant improvement.
5. **Choose the best rule.** Compare all legal candidates using the requested algorithm's criterion.
6. **Repeat recursively.** Build child trees until a stopping condition is met; then predict by following the tests.

7.2 How to write a prediction function

Training builds a nested structure. Prediction is simpler:

1. Start at the root node.
2. Read the feature named by the node.
3. Evaluate the node's test.
4. Move to the matching child.
5. Repeat until a leaf is reached.
6. Return the leaf's class or numeric value.

7.3 A study sequence that builds understanding

1. **Day 1 - Entropy only.** Calculate entropy for pure, balanced and imbalanced nodes by hand. Practise the logarithms and class proportions.
2. **Day 2 - ID3 only.** Recalculate all four information gains in the Play Tennis example. Draw the root and complete the Sunny and Rain branches by hand.
3. **Day 3 - C4.5 only.** Calculate split information and gain ratio. Repeat the Score threshold example. Explain why the raw gain and gain ratio are different quantities.

4. **Day 4 - CART classification only.** Calculate Gini for a parent and two children. Practise binary category groupings and numeric midpoints.
5. **Day 5 - CART regression only.** Calculate means and SSE for candidate splits. Check that each leaf predicts the mean of its target values.
6. **Day 6 - Code reconstruction.** Write entropy, Gini, partitioning and majority-class functions from memory. Then implement one training algorithm without looking at the notes.
7. **Day 7 - Mixed practice.** For each question, identify the algorithm first, write its criterion from memory, solve the root by hand, and only then draw the tree.

7.4 A code-reconstruction drill

Practise each function independently before writing the full tree:

1. Write `entropy(rows, target)`. Test it on labels `[Yes, Yes, No, No]` and verify that it returns 1 bit.
2. Write `gini(rows, target)`. Test the same balanced labels and verify that it returns 0.5.
3. Write a function that groups rows by one categorical feature.
4. Write a function that divides rows using `feature <= threshold`.
5. Write weighted child impurity without recursion.
6. Write the recursive builder with only two stopping conditions: pure node and no usable split.
7. Add prediction only after training works.
8. Add depth and minimum-size controls last.

7.5 Questions to ask yourself while debugging

- Does every row go into exactly one child?
- Do the child counts add back to the parent count?
- Are child weights calculated using the parent size?
- Am I using the correct formula for this algorithm?
- Does a pure child have zero entropy and zero Gini?
- Does the algorithm stop when a node is pure?
- Does the prediction function use exactly the same test that training stored?

8 Additional practice problems

Attempt these before reading any worked solution. For every problem, write the formula first, keep enough decimal places during intermediate calculations, and round only at the end.

Problem A - Entropy

A node contains 12 positive and 4 negative examples. Calculate its entropy. Then calculate the entropy if a split creates one pure child of size 6 and another child with 6 positive and 4 negative examples. Find the information gain.

Problem B - ID3 root selection

A dataset contains 10 rows with 6 Yes and 4 No. Attribute A creates children (4 Yes, 0 No) and (2 Yes, 4 No). Attribute B creates children (3 Yes, 2 No) and (3 Yes, 2 No). Calculate the information gain for both and choose the ID3 root.

Problem C - C4.5 comparison

Two candidate attributes have the following values:

	Gain	SplitInfo
A	0.20	1.00
B	0.18	0.60

Calculate both gain ratios. Which is larger? Then explain why a practical C4.5 selection procedure also considers average information gain.

Problem D - CART classification

A parent has 8 Yes and 2 No examples. A candidate split produces child 1 with 4 Yes and 0 No, and child 2 with 4 Yes and 2 No. Calculate the parent Gini, weighted child Gini and Gini reduction.

Problem E - CART regression

A parent target vector is $(1, 2, 3, 8, 9, 10)$. Calculate its mean and SSE. Evaluate the split $(1, 2, 3)$ versus $(8, 9, 10)$. What value does each leaf predict?

Problem F - Conceptual distinctions

For each statement, identify whether it describes ID3, C4.5 or CART:

1. A categorical feature creates one branch for every value and the split maximizes information gain.
2. Numeric thresholds are evaluated using gain ratio.
3. Every internal node has exactly two children.
4. A regression leaf predicts the mean target value and splits minimize squared error.
5. Cost-complexity pruning penalizes the number of leaves.

9 Answer checks for the additional practice

These are compact checks, not substitutes for writing the full method.

1. **Problem A.** Parent entropy is $-\frac{12}{16}\log_2\frac{12}{16} - \frac{4}{16}\log_2\frac{4}{16} \approx 0.8113$. The first child is pure, so its entropy is 0. The second child has proportions 0.6 and 0.4, so its entropy is approximately 0.9710. Weighted child entropy is $\frac{6}{16}(0) + \frac{10}{16}(0.9710) \approx 0.6069$. Gain is approximately 0.2044.
2. **Problem B.** Parent entropy is approximately 0.9710. Attribute A has weighted child entropy $\frac{4}{10}(0) + \frac{6}{10}(0.9183) = 0.5510$, giving gain approximately 0.4200. Attribute B has two children each with entropy 0.9710, so its gain is 0. Attribute A is selected.
3. **Problem C.** Ratio A is 0.20. Ratio B is 0.30. B has the larger ratio, but the average-gain safeguard matters because a candidate with very low raw gain may otherwise obtain an attractive ratio from a small denominator.

4. **Problem D.** Parent Gini is $1-(8/10)^2-(2/10)^2=0.32$. Child 1 Gini is 0. Child 2 Gini is $1-(4/6)^2-(2/6)^2=4/9\approx 0.4444$. Weighted child Gini is $\frac{4}{10}(0)+\frac{6}{10}(0.4444)\approx 0.2667$. Reduction is approximately 0.0533.
5. **Problem E.** Mean is 5.5 and parent SSE is 97.5. Each child has mean 2 and 9, respectively; each child SSE is 2, so total child SSE is 4. The left leaf predicts 2 and the right leaf predicts 9.
6. **Problem F.** (1) ID3; (2) C4.5; (3) CART; (4) CART; (5) CART.

10 Final examination checklist

Before submitting a decision-tree numerical answer, check the following:

☐ I identified the requested algorithm before calculating.

☐ I wrote the correct parent class counts or target values.

☐ I used the correct impurity measure: entropy, gain ratio, Gini or SSE.

☐ I weighted child impurities by child size divided by parent size.

☐ I compared all relevant candidate splits, not just one.

☐ I stated whether the winning split is multiway or binary.

☐ I showed the child subsets and continued recursively where required.

☐ I used a majority class for a classification leaf or a mean for a squared-error regression leaf.

☐ I did not confuse pre-pruning, post-pruning and stopping conditions.

Key idea

The mental model to remember
A decision tree repeatedly asks a question that makes the remaining data more predictable. ID3 measures the reduction in entropy. C4.5 adjusts that information gain using split information and adds support for practical data conditions. CART insists on binary questions and measures classification impurity with Gini or regression error with squared error. Once the data partitioning, weighted scoring and recursive repetition are understood, the code is a direct translation of the mathematics.